

KEE Platform

Database Schema Documentation

PostgreSQL 15 · Prisma ORM · Multi-tenant SaaS schema
Generated 2026-07-17

Covers: relational document storage, soft delete, referential integrity

1. Table of Contents

- 2. Overview & Conventions
- 3. Entity-Relationship Diagram
- 4. Table Reference (all 15 models)
- 5. Sample Records (live, redacted)
- 6. Migration Guidelines
- 7. Applied Migration History

2. Overview & Conventions

The KEE platform is a multi-tenant SaaS application. Each tenant is a "Shop" (a key-duplication business); Shop Admin users operate within a single shop, while Super Admin users operate across all shops. The schema is defined in `prisma/schema.prisma` and managed via Prisma Migrate against PostgreSQL.

2.1 Soft delete

Rows in soft-deletable tables are never physically removed by application code. Instead, a nullable `deletedAt` timestamp column marks a row as deleted. A Prisma Client Extension (`src/tenant/tenant.service.ts`) intercepts every query against these models: `delete/deleteMany` calls are transparently rewritten into `update/updateMany` calls that set `deletedAt`, and all read operations automatically get `deletedAt: null` merged into their `where` clause unless the caller already specified a `deletedAt` filter. Soft-deletable models: `Shop`, `User`, `MasterKey`, `Customer`, `CustomerDocument`, `ShopDocument`, `Product`.

Important limitation: this filtering only applies to the top-level queried model. Nested `include/select` relations are resolved by Prisma via a separate join/query that the extension does not intercept, so a soft-deleted related row can still surface inside a nested include unless the caller adds an explicit `where: { deletedAt: null }` on that nested relation. See §6 for the concrete pattern.

2.2 Tenant (shopId) row-level scoping

For a subset of models (`Customer`, `Subscription`, `User`, `ActivityLog`, `Notification`), the same extension automatically scopes all queries to the current request's `shopId` when a Shop Admin request context is active (read via `AsyncLocalStorage`, set by `TenantInterceptor` on every request). Super Admin requests are not row-scoped by this mechanism; those services pass explicit filters where shop-specific scoping is needed.

2.3 Document storage

Files (shop verification documents, customer ID proofs) are stored on disk/object storage and referenced by pointer (`fileUrl`, `fileKey`, `fileSize`) in dedicated relational tables (`ShopDocument`, `CustomerDocument`) rather than embedded as base64 inside JSON columns. This refactor specifically eliminated the legacy pattern where `Shop.companyDetails` held base64-encoded document blobs directly in a JSON text column — see §5 and the migration report (`docs/MIGRATION_REPORT.md`) for details of the data migration that moved existing records.

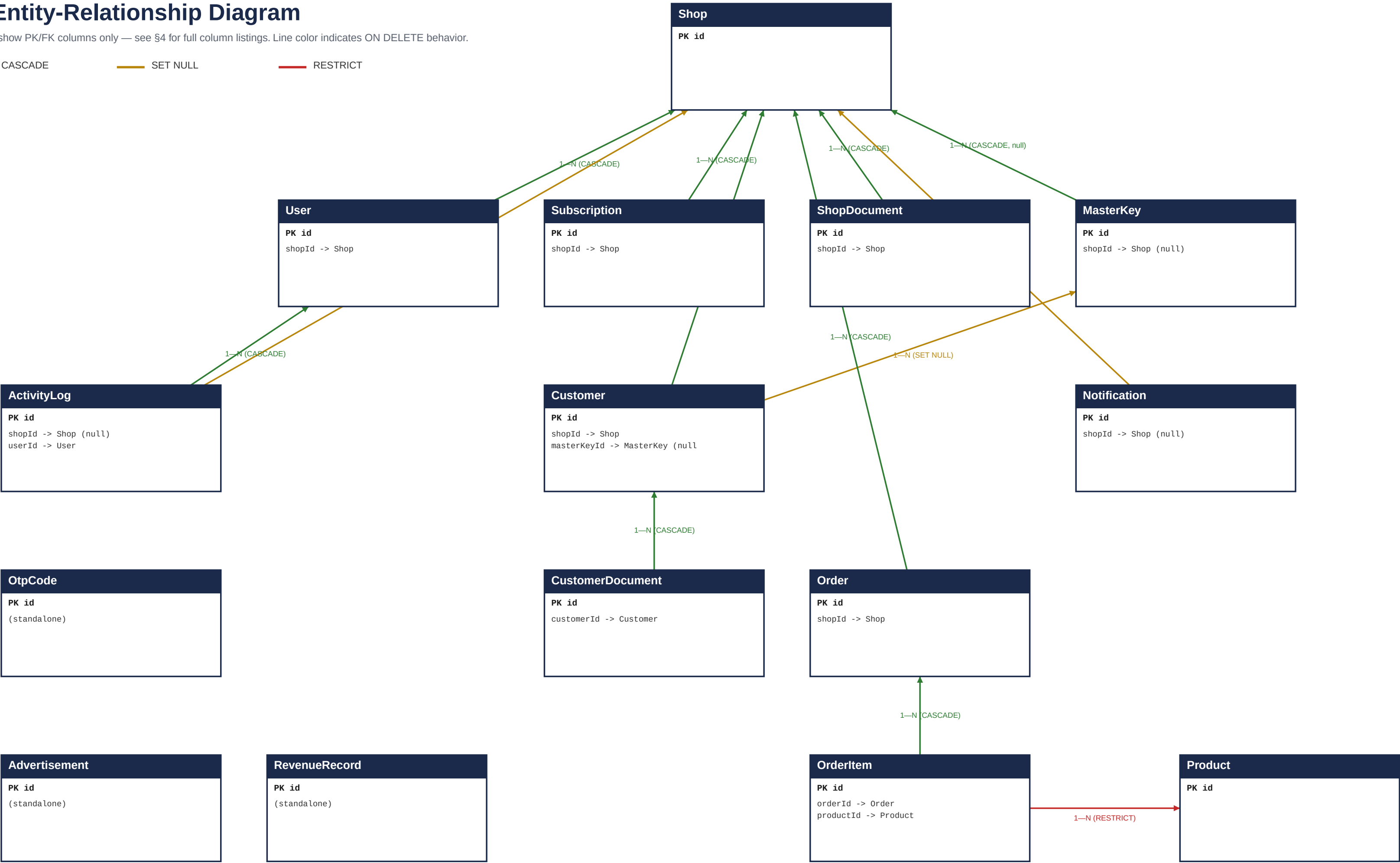
2.4 Identifiers

All primary keys are UUID strings generated client-side by Prisma (`uuid()`). No auto-incrementing integer IDs are used, which avoids ID-guessing/enumeration concerns and works cleanly with the eventual possibility of multi-region / offline-first writes.

3. Entity-Relationship Diagram

Boxes show PK/FK columns only — see §4 for full column listings. Line color indicates ON DELETE behavior.

CASCADE SET NULL RESTRICT



4. Table Reference

Full column, index, and relationship listing for all 15 tables, in the order they appear in `prisma/schema.prisma`.

Shop

SOFT-DELETE

A tenant. Represents a single key-duplication shop on the platform. The root of the multi-tenant hierarchy — most other tables hang off a `shopId`, directly or transitively.

Column	Type	Notes
id	String (uuid)	PK
name	String	
logoUrl	String?	nullable
companyDetails	String?	nullable, JSON text (scalar business fields only)
themeColor	String	default '#9C27B0'
isActive	Boolean	default true
isApproved	Boolean	default false
storageUsed	BigInt	default 0, bytes
deletedAt	DateTime?	soft delete marker
createdAt	DateTime	default now()
updatedAt	DateTime	auto-updated

Indexes: deletedAt

Relations:

users	1–N	User	(User.shopId, ON DELETE CASCADE)
subscriptions	1–N	Subscription	(Subscription.shopId, ON DELETE CASCADE)
customers	1–N	Customer	(Customer.shopId, ON DELETE CASCADE)
documents	1–N	ShopDocument	(ShopDocument.shopId, ON DELETE CASCADE)
masterKeys	1–N	MasterKey	(MasterKey.shopId, ON DELETE CASCADE, nullable FK)
orders	1–N	Order	(Order.shopId, ON DELETE CASCADE)
activityLogs	1–N	ActivityLog	(ActivityLog.shopId, ON DELETE SET NULL, nullable FK)
notifications	1–N	Notification	(Notification.shopId, ON DELETE SET NULL, nullable FK)

User

SOFT-DELETE · TENANT-SCOPED

An authenticated account: either a `SUPER_ADMIN` (platform operator, `shopId` null) or a `SHOP_ADMIN` (scoped to exactly one shop).

Column	Type	Notes
id	String (uuid)	PK
email	String	UNIQUE
passwordHash	String	bcrypt hash
name	String	
role	Role enum	SUPER_ADMIN SHOP_ADMIN
shopId	String?	FK -> Shop.id, nullable (null for Super Admin)
deletedAt	DateTime?	soft delete marker
createdAt	DateTime	default now()
updatedAt	DateTime	auto-updated

Indexes: shopId | deletedAt

Relations:
shop N-1 Shop (shopId, ON DELETE CASCADE)
activityLogs 1-N ActivityLog (ActivityLog.userId, ON DELETE CASCADE)

Subscription

TENANT-SCOPED

A shop's billing plan period. Multiple rows accumulate over time; only the most recent is ACTIVE.

Column	Type	Notes
id	String (uuid)	PK
shopId	String	FK -> Shop.id
plan	Plan enum	TRIAL MONTHLY YEARLY
status	SubscriptionStatus enum	default ACTIVE; ACTIVE SUSPENDED EXPIRED
startDate	DateTime	
endDate	DateTime	
createdAt	DateTime	default now()
updatedAt	DateTime	auto-updated

Indexes: shopId | (shopId, status)

Relations:
shop N-1 Shop (shopId, ON DELETE CASCADE)

MasterKey

SOFT-DELETE

A key-blank catalog entry. shopId null = global/legacy catalog entry visible to all shops; shopId set = shop-specific key created during customer registration.

Column	Type	Notes
id	String (uuid)	PK
keyNumber	String	UNIQUE with shopId
brand	String	
category	String	
blankNumber	String	
frontImageUrl	String?	nullable
backImageUrl	String?	nullable
description	String?	nullable
notes	String?	nullable
status	KeyStatus enum	default ACTIVE; ACTIVE INACTIVE
shopId	String?	FK -> Shop.id, nullable
deletedAt	DateTime?	soft delete marker
createdAt	DateTime	default now()
updatedAt	DateTime	auto-updated

Indexes: UNIQUE (shopId, keyNumber) | shopId | deletedAt

Relations:
shop N-1 Shop (shopId, ON DELETE CASCADE, nullable)
customers 1-N Customer (Customer.masterKeyId, ON DELETE SET NULL, nullable)

Customer

SOFT-DELETE · TENANT-SCOPED

A key-duplication customer record captured by a shop admin. idProofNumber is encrypted at rest (application-level, not column-level encryption).

Column	Type	Notes
id	String (uuid)	PK
shopId	String	FK -> Shop.id
name	String	
phone	String	
address	String?	nullable
idProofType	String?	nullable
idProofNumber	String?	nullable, encrypted at rest
reason	String?	nullable
keyNumber	String	
vehicleNumber	String?	nullable
masterKeyId	String?	FK -> MasterKey.id, nullable
latitude / longitude	Float?	nullable
mapsLink	String?	nullable
capturedAddress	String?	nullable
photoUrl / signatureUrl	String?	nullable
deletedAt	DateTime?	soft delete marker
createdAt	DateTime	default now()
updatedAt	DateTime	auto-updated

Indexes: shopId | masterKeyId | (shopId, deletedAt) | (shopId, phone)

Relations:

shop	N-1	Shop	(shopId, ON DELETE CASCADE)
masterKey	N-1	MasterKey	(masterKeyId, ON DELETE SET NULL, nullable)
documents	1-N	CustomerDocument	(CustomerDocument.customerId, ON DELETE CASCADE)

CustomerDocument

SOFT-DELETE

A file attached to a customer record (ID proof scans, etc). Nested `include`s of this relation MUST filter deletedAt: null explicitly — see docs/DEVELOPER_GUIDE.md §4.

Column	Type	Notes
id	String (uuid)	PK
customerId	String	FK -> Customer.id
documentType	String	free-form, not an enum
fileUrl	String	
fileKey	String	
fileSize	Int	bytes
deletedAt	DateTime?	soft delete marker
createdAt	DateTime	default now()
updatedAt	DateTime	auto-updated

Indexes: customerId | deletedAt

Relations:

customer	N-1	Customer	(customerId, ON DELETE CASCADE)
----------	-----	----------	---------------------------------

ShopDocument

SOFT-DELETE

A file attached to a shop record (shop photo, business license, owner's ID). Introduced by this refactor to replace the legacy Shop.companyDetails.{shopPhoto,shopLicense,ownerAadhaar} base64-embedded JSON fields. Mirrors CustomerDocument.

Column	Type	Notes
id	String (uuid)	PK
shopId	String	FK -> Shop.id
documentType	String	free-form; conventional values 'SHOP_PHOTO' 'SHOP_LICENSE' 'OWNER_AADHAAR'
fileUrl	String	
fileKey	String	
fileSize	Int	default 0, bytes
deletedAt	DateTime?	soft delete marker
createdAt	DateTime	default now()
updatedAt	DateTime	auto-updated

Indexes: shopId | (shopId, documentType) | deletedAt

Relations:

shop N-1 Shop (shopId, ON DELETE CASCADE)

OtpCode

A one-time-password issued for a verification purpose (login/registration/reset). Standalone — no FKs.

Column	Type	Notes
id	String (uuid)	PK
identifier	String	email/phone the OTP was sent to
method	String	e.g. 'EMAIL' 'SMS'
purpose	String	e.g. 'LOGIN' 'RESET_PASSWORD'
codeHash	String	hashed, never stored plaintext
consumed	Boolean	default false
expiresAt	DateTime	
createdAt	DateTime	default now()

Indexes: (identifier, purpose)

Advertisement

A platform-wide or shop-targeted promotional banner/popup/notice. Standalone — no FKs (targetShops is a raw ID array, not a relation).

Column	Type	Notes
id	String (uuid)	PK
title	String	
imageUrl	String	
type	AdType enum	POPUP BANNER NOTICE
startDate / endDate	DateTime	
priority	Int	default 0
targetAll	Boolean	default true
targetShops	String[]	raw shop-ID array, no FK constraint
createdAt	DateTime	default now()

updatedAt

DateTime

auto-updated

ActivityLog

TENANT-SCOPED

An immutable audit-trail entry. Never soft- or hard-deleted by application code (no deletedAt column at all) — audit history must outlive the rows it describes.

Column	Type	Notes
id	String (uuid)	PK
shopId	String?	FK -> Shop.id, nullable
userId	String	FK -> User.id
action	String	e.g. 'LOGIN', 'CUSTOMER_CREATE'
details	String	JSON string or description, PII masked
ipAddress	String?	nullable
createdAt	DateTime	default now()

Indexes: shopId | userId | action

Relations:

shop	N-1	Shop	(shopId, ON DELETE SET NULL, nullable)
user	N-1	User	(userId, ON DELETE CASCADE)

Notification

TENANT-SCOPED

A shop-specific or system-wide (shopId null) announcement/alert.

Column	Type	Notes
id	String (uuid)	PK
shopId	String?	FK -> Shop.id, nullable (null = global)
title	String	
message	String	
type	String	e.g. 'SUB_EXPIRY', 'NEW_KEY'
isRead	Boolean	default false
createdAt	DateTime	default now()

Indexes: shopId

Relations:

shop	N-1	Shop	(shopId, ON DELETE SET NULL, nullable)
------	-----	------	--

RevenueRecord

A manually-entered platform revenue figure for a given month/year (Super Admin reporting). Standalone — no FKs.

Column	Type	Notes
id	String (uuid)	PK
month	Int	
year	Int	
amount	Float	
notes	String?	nullable
createdAt	DateTime	default now()
updatedAt	DateTime	auto-updated

Product

SOFT-DELETE

A catalog item sellable to shops (key blanks, machines, accessories).

Column	Type	Notes
id	String (uuid)	PK
name	String	
description	String?	nullable
price	Float	
discountPercentage	Float	default 0
offerPrice	Float?	nullable, computed
imageUrl	String?	nullable
category	String	'Key Blanks' 'Machines' 'Accessories'
stock	Int	default 0
status	KeyStatus enum	default ACTIVE; ACTIVE INACTIVE
deletedAt	DateTime?	soft delete marker
createdAt	DateTime	default now()
updatedAt	DateTime	auto-updated

Indexes: deletedAt

Relations:

orderItems 1–N OrderItem (OrderItem.productId, ON DELETE RESTRICT)

Order

A shop purchase order for platform products. Transactional/financial record — intentionally has no deletedAt; only status-transitioned, never deleted.

Column	Type	Notes
id	String (uuid)	PK
shopId	String	FK -> Shop.id
totalAmount	Float	
status	String	default 'PENDING'; PENDING DISPATCHED DELIVERED CANCELLED
createdAt	DateTime	default now()
updatedAt	DateTime	auto-updated

Indexes: shopId | (shopId, status)

Relations:

shop items N–1 Shop 1–N OrderItem (shopId, ON DELETE CASCADE)
(OrderItem.orderId, ON DELETE CASCADE)

OrderItem

A line item within an order. productId is ON DELETE RESTRICT — a product with order history cannot be hard-deleted (ProductService.deleteProduct soft-deletes via status=INACTIVE instead).

Column	Type	Notes
id	String (uuid)	PK
orderId	String	FK -> Order.id
productId	String	FK -> Product.id
quantity	Int	
price	Float	price at time of order (snapshot)

Indexes: orderId | productId

Relations:

order	N-1	Order	(orderId, ON DELETE CASCADE)
product	N-1	Product	(productId, ON DELETE RESTRICT)

5. Sample Records (live, redacted)

The following records were pulled live from the development database at documentation-generation time to illustrate real row shapes. Sensitive fields (password hashes, encrypted PII) are redacted.

Shop

```
id          : 9062ddab-fb24-4160-9939-f7369fb8dc23
name        : Doc Verify Shop
logoUrl     : null
companyDetails : <redacted>
themeColor  : #8C24FF
isActive    : true
isApproved  : true
storageUsed  : 0
deletedAt   : null
createdAt   : 2026-07-16T20:58:43.050Z
updatedAt   : 2026-07-17T06:33:56.842Z
```

User

```
id          : 2afe4b48-6bdb-4e71-a9a0-ceef3ea0398e
email       : docverify_test@example.com
passwordHash : <redacted>
name        : Doc Owner
role        : SHOP_ADMIN
shopId      : 9062ddab-fb24-4160-9939-f7369fb8dc23
deletedAt   : null
createdAt   : 2026-07-16T20:58:43.060Z
updatedAt   : 2026-07-16T20:58:43.060Z
```

Customer

```
id          : 53680906-42ed-4347-aeb2-d72f1159653e
shopId      : 299da04c-c11e-474d-8e09-7f7a49af40eb
name        : Curl Verify Customer
phone       : 9876500001
address     : null
idProofType : null
idProofNumber : <redacted>
reason      : null
keyNumber   : VERIFYTEST1
vehicleNumber : null
masterKeyId : null
latitude    : 28.6139
longitude   : 77.209
mapsLink    : https://www.google.com/maps?q=28.6139,77.209
capturedAddress : Connaught Place, New Delhi, India
photoUrl    : null
signatureUrl : null
deletedAt   : null
createdAt   : 2026-07-16T20:27:47.454Z
updatedAt   : 2026-07-16T20:27:47.454Z
```

ShopDocument

```
id          : 7300c742-1df6-4b24-acb6-d570af1e184b
shopId      : 9062ddab-fb24-4160-9939-f7369fb8dc23
documentType : SHOP_PHOTO
fileUrl      : /api/uploads/9062ddabfb2441609939f7369fb8dc23_1784270036826_ooilp3t.png
fileKey      : 9062ddabfb2441609939f7369fb8dc23_1784270036826_ooilp3t.png
fileSize     : 68
deletedAt    : null
createdAt    : 2026-07-17T06:33:56.829Z
updatedAt    : 2026-07-17T06:33:56.829Z
```

MasterKey

```
id          : e338579e-3d72-4bcb-8c4f-82f98daa4a92
keyNumber   : TN0989
brand       : Manual Input
category    : Vehicle Keys
blankNumber : TN0989
frontImageUrl : null
backImageUrl : null
description : null
notes       : null
status      : ACTIVE
shopId      : 299da04c-c11e-474d-8e09-7f7a49af40eb
deletedAt   : null
createdAt   : 2026-07-16T20:22:23.455Z
updatedAt   : 2026-07-16T20:22:23.455Z
```

Subscription

id	: b808c33b-bdd6-4aed-8ac6-04bc014730d1
shopId	: 9062ddab-fb24-4160-9939-f7369fb8dc23
plan	: TRIAL
status	: ACTIVE
startDate	: 2026-07-16T20:58:43.069Z
endDate	: 2026-07-23T20:58:43.069Z
createdAt	: 2026-07-16T20:58:43.071Z
updatedAt	: 2026-07-16T20:58:43.071Z

6. Migration Guidelines

6.1 Adding soft delete to a new model

- Add `deletedAt DateTime?` to the model, plus `@@index([deletedAt])` (or a composite index if tenant-scoped).
- Add the model name to `SOFT_DELETE_MODELS` in `src/tenant/tenant.service.ts`.
- Run `npx prisma migrate dev --name <description>` to generate and apply the migration.
- Audit every existing nested `include/select` of this model elsewhere in the codebase and add an explicit `where: { deletedAt: null }` filter on the relation — the extension does not filter nested includes automatically (see §2.1).

6.2 Adding tenant scoping to a new model

- Add the model name to `TENANT_SCOPED_MODELS` in `src/tenant/tenant.service.ts`.
- Check whether the create-input auto-injection logic needs a model-specific branch (some models need relation-connect shape instead of a bare `shopId` scalar — see the code comment in the extension's create branch).
- Add a model-specific where-merge branch if the model doesn't want simple `shopId` equality (e.g. Notification's shop-specific-OR-global OR clause).

6.3 Document storage convention

New document-attachment features should follow the `ShopDocument/CustomerDocument` shape: `id`, owning-entity FK, `documentType` (free-form string), `fileUrl`, `fileKey`, `fileSize`, `deletedAt`, timestamps. Do not add new base64-embedded-in-JSON document fields.

6.4 Testing checklist

- Run `npx tsc --noEmit` and `npx jest` before and after any schema/query change.
- For tenant-scoping or soft-delete changes, manually verify against a running dev server: cross-tenant isolation, soft-delete-then-refetch exclusion (including derived counts), and both the Shop Admin and Super Admin read paths (they are separate methods that can drift).

7. Applied Migration History

2 migration(s) found under prisma/migrations/, applied in order:

1. 20260717113455_baseline
2. 20260717113900_shop_documents_soft_delete_and_indexes

To apply pending migrations against a target database: `cd backend && npx prisma migrate deploy`. To generate a new migration during development: `npx prisma migrate dev --name <description>`. Always run `npx prisma migrate status` before deploying to confirm the target database is in the expected state.

The one-off data migration for legacy embedded shop documents (`scripts/migrate-shop-documents.ts`) is separate from schema migrations — it is an idempotent, re-runnable data-fixup script, not a Prisma migration, and only needs to be run once per environment after the `ShopDocument` table exists. See `docs/MIGRATION_REPORT.md §3` for full details and execution results.